ПРАКТИЧНЕ ЗАНЯТТЯ №7

Тема: Параметричний поліморфізм. Callback-функції

Питання для самостійного опрацювання

Виконав: Вівчар Вадим Вікторович
Група: АЛК-43
Дисципліна: Об'єктно орієнтоване програмування

У звіті наведено відповіді на контрольні запитання за темою callback-функцій та класів-інтерфейсів. Матеріал підготовлено за лекцією про callback-функції і класами-інтерфейсами у C++, де розглядаються призначення інтерфейсів, їх канонічні властивості, особливості реалізації в стандартному C++ та специфіка Visual C++.

Короткі теоретичні відомості

Класи-інтерфейси використовують для роз'єднання інтерфейсу та реалізації. Такий підхід дає змогу будувати гнучкі архітектурні рішення, зменшувати зв'язність модулів і задавати для похідних класів чіткий контракт поведінки.

У лекційному матеріалі клас-інтерфейс описується як базовий абстрактний клас, який містить тільки чисто віртуальні функції, не має полів, конструкторів та деструктора у канонічному варіанті. Похідний клас повинен реалізувати всі методи інтерфейсу, якщо на його основі планується створювати об'єкти.

Ідея callback-функцій у C++ тісно пов'язана з інтерфейсами. Замість передавання окремої функції можна передати вказівник на об'єкт класу-інтерфейсу, а виклик необхідних дій у нижчому модулі організувати через поліморфні виклики віртуальних методів.

Ілюстративний приклад з лекції

```
class IExecutable
{
public:
    virtual int start() = 0;
    virtual void run() = 0;
    virtual void end() = 0;
};

class Work : public IExecutable
{
public:
    int start() override { return 1; }
    void run() override { }
    void end() override { }
};

class Game : public IExecutable
{
public:
    int start() override { return 1; }
    void run() override { }
    void end() override { }
};
```

У цьому прикладі IExecutable задає лише інтерфейс роботи, а класи Work і Game реалізують його власним способом. Саме така побудова використовується для пояснення сутності інтерфейсів у лекції.

1. Що містить клас-інтерфейс?

Клас-інтерфейс містить набір методів, які утворюють контракт для похідних класів. У класичному вигляді він містить тільки чисто віртуальні функції без реалізації. Отже, інтерфейс описує не внутрішній стан об'єкта,

а перелік дій, які повинні підтримувати всі класи-реалізатори.

Наприклад, інтерфейс IExecutable може містити методи start(), run() та end(). Тоді будь-який клас, що реалізує цей інтерфейс, зобов'язаний забезпечити виконання кожного з цих методів.

2. Чого не повинно бути у класі-інтерфейсі згідно з канонами ООП?

Згідно з канонами ООП, у класі-інтерфейсі не повинно бути полів даних, конструкторів, деструктора та готової реалізації методів. Усі функції інтерфейсу повинні бути public і чисто віртуальними. Такий підхід забезпечує максимальну абстрактність і не нав'язує похідним класам конкретну реалізацію або стан.

Іншими словами, інтерфейс відповідає на питання "що має вміти об'єкт", але не визначає "як саме це буде реалізовано".

3. Як класи-інтерфейси можна реалізувати на C++?

У стандартному C++ класи-інтерфейси реалізують за допомогою абстрактного базового класу, в якому всі методи оголошені як чисто віртуальні, тобто мають форму = 0. Похідні класи успадковуються від такого базового класу і реалізують усі потрібні методи.

Отже, окремого універсального ключового слова interface у стандартному C++ немає, тому інтерфейс моделюється дисциплінованим використанням абстрактних класів.

4. Чи може бути в класі-інтерфейсі віртуальний деструктор?

У канонічному теоретичному визначенні класу-інтерфейсу деструктора бути не повинно. Проте у практиці C++ в абстрактному базовому класі-інтерфейсі часто додають віртуальний деструктор, щоб забезпечити коректне знищення похідних об'єктів через вказівник на базовий тип.

Тобто в чистій теорії інтерфейс не містить деструктора, але в реальному програмуванні C++ віртуальний деструктор часто є необхідним із міркувань безпеки керування пам'яттю.

5. Які ключові слова можна використати в реалізації Visual C++ для оголошення класів-інтерфейсів?

У Visual C++ для оголошення інтерфейсів можна зустріти ключові слова interface та __interface. У лекції зазначено, що interface використовується в середовищі .Net Framework, а __interface - для native C++.

При цьому __interface спрощує запис: не потрібно окремо вказувати public і virtual для кожного методу, оскільки функції автоматично вважаються

публічними і чисто віртуальними.

6. Чи можна створити об'єкт типу класу-інтерфейсу?

Ні, об'єкт типу класу-інтерфейсу створити не можна. Причина в тому, що інтерфейс є абстрактним типом і не має повної реалізації всіх своїх методів. Створювати можна лише об'єкти похідних класів, які реалізували всі функції інтерфейсу.

Отже, інтерфейс існує як засіб опису контракту, а не як повноцінний тип для безпосереднього інстанціювання.

7. У чому відмінність класів-інтерфейсів від абстрактних класів?

Клас-інтерфейс є більш суворою формою абстракції. Він містить тільки контракт і не має стану чи реалізації. Абстрактний клас, навпаки, може містити не лише чисто віртуальні методи, а й звичайні методи з реалізацією, поля даних, допоміжну логіку та часткову спільну поведінку для похідних класів.

Отже, інтерфейс призначений для задання єдиного набору вимог, а абстрактний клас часто використовується і для часткового повторного використання коду.

8. Чи можна назвати клас-інтерфейс найвищим рівнем абстракції в ООП?

Так, клас-інтерфейс можна вважати одним із найвищих рівнів абстракції в ООП. Він повністю відокремлює опис поведінки від її реалізації та не містить конкретного стану об'єкта. Саме тому інтерфейс задає максимально чистий контракт взаємодії.

Проте на практиці варто пам'ятати, що в C++ інтерфейс реалізується через абстрактний клас, тому ця абстракція досягається не окремим універсальним механізмом мови, а певною дисципліною проектування.

Приклад використання __interface у Visual C++

```
__interface IExecutable
{
    int start();
    void run();
    void end();
};
```

Лекція показує, що __interface робить запис більш канонічним, але водночас створює обмеження. Зокрема, при спробі додати віртуальний деструктор до __interface компілятор Visual C++ видає помилку, оскільки такий інтерфейс не може містити деструктор.

Порівняння інтерфейсу та абстрактного класу

Ознака	Клас-інтерфейс	Абстрактний клас
Методи	Лише чисто віртуальні	Можуть бути і чисто віртуальні, і звичайні
Поля даних	Не повинно бути	Можуть бути
Конструктор	Не повинен використовуватись	Може бути
Деструктор	У канонічному варіанті відсутній	Може бути, часто віртуальний
Призначення	Задати контракт	Задати контракт і часткову реалізацію

Висновок

У лекційному матеріалі класи-інтерфейси розглядаються як важливий інструмент побудови слабко зв'язаних систем, у яких чітко відокремлюється контракт від реалізації.

Для стандартного C++ інтерфейс не є окремим універсальним синтаксичним елементом, тому він моделюється за допомогою абстрактного базового класу з чисто віртуальними методами. У Visual C++ існують спеціальні ключові слова interface та __interface, але вони мають власні обмеження.

Відповіді на контрольні запитання підтверджують, що клас-інтерфейс задає максимально високий рівень абстракції і є зручним інструментом для побудови гнучкої архітектури програмних систем.

Додаток А. Практичні приклади у Visual Studio

Нижче наведено приклади реалізації класу-інтерфейсу та callback-виклику в середовищі Microsoft Visual Studio.

1. Клас-інтерфейс IExecutable та його реалізація класами Work і Game

Microsoft Visual Studio

```
#include <iostream>
using namespace std;

class IExecutable {
public:
    virtual int start() = 0;
    virtual void run() = 0;
    virtual void end() = 0;

    virtual ~IExecutable() {
        cout << "IExecutable destructor" << endl;
    }
};

class Work : public IExecutable {
public:
    int start() override {
        cout << "Work started" << endl;
        return 1;
    }

    void run() override {
        cout << "Work is running" << endl;
    }

    void end() override {
        cout << "Work ended" << endl;
    }

    ~Work() {
        cout << "Work destructor" << endl;
    }
};

class Game : public IExecutable {
public:
    int start() override {
        cout << "Game started" << endl;
        return 1;
    }

    void run() override {
        cout << "Game is running" << endl;
    }

    void end() override {
        cout << "Game ended" << endl;
    }

    ~Game() {
        cout << "Game destructor" << endl;
    }
};

int main() {
    IExecutable* obj1 = new Work();
    IExecutable* obj2 = new Game();

    obj1->start();
    obj1->run();
    obj1->end();

    cout << endl;

    obj2->start();
    obj2->run();
    obj2->end();

    delete obj1;
    delete obj2;

    return 0;
}
```

Рисунок 1 - Код програми з демонстрацією класу-інтерфейсу та його реалізації.

2. Результат виконання програми з класами Work і Game

```
Microsoft Visual Studio Debug

Work started
Work is running
Work ended

Game started
Game is running
Game ended
Work destructor
IExecutable destructor
Game destructor
IExecutable destructor

E:\user vadim\Don\Tema7\x64\Debug\Tema7.exe (process 18344) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the consol
when debugging stops.
Press any key to close this window . . .
```

Рисунок 2 - Результат виконання програми з поліморфними викликами через інтерфейс IExecutable.

3. Callback-виклик через функцію executeCallback()

Microsoft Visual Studio

```cpp
#include <iostream>
using namespace std;

class IExecutable {
public:
    virtual int start() = 0;
    virtual void run() = 0;
    virtual void end() = 0;

    virtual ~IExecutable() {}
};

class DownloadTask : public IExecutable {
public:
    int start() override {
        cout << "DownloadTask started" << endl;
        return 1;
    }

    void run() override {
        cout << "Downloading data..." << endl;
    }

    void end() override {
        cout << "DownloadTask finished" << endl;
    }
};

class BackupTask : public IExecutable {
public:
    int start() override {
        cout << "BackupTask started" << endl;
        return 1;
    }

    void run() override {
        cout << "Creating backup..." << endl;
    }

    void end() override {
        cout << "BackupTask finished" << endl;
    }
};

void executeCallback(IExecutable* task) {
    if (task->start()) {
        task->run();
        task->end();
    }
}

int main() {
    DownloadTask d;
    BackupTask b;

    executeCallback(&d);
    cout << endl;
    executeCallback(&b);

    return 0;
}
```

Рисунок 3 - Код програми з демонстрацією callback-виклику через інтерфейс.

4. Результат виконання callback-програми

```
Microsoft Visual Studio Debug

DownloadTask started
Downloading data...
DownloadTask finished

BackupTask started
Creating backup...
BackupTask finished

E:\user vadim\Don\Tema6.1\x64\Debug\Tema6.1.exe (process 16668) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 4 - Результат виконання програми з callback-викликом.